YSP / 1.0

Yunsee Stream Protocol

数据报传输与分层对象封装规范

文档编号	YSP-1.0 / 公开版
状态	稳定
默认端口	UDP/46000

YSP/1 在 UDP 上提供会话协商、对象元数据、乱序分片、重传、完整性校验，以及可组合的多层内容变换。本规范包含实现兼容接收端所需的全部线上格式与算法。

1. 记号与通用规则

- 所有多字节整数均为无符号整数，使用网络字节序（big-endian）。
- 字符串使用 UTF-8，不包含结尾 NUL。
- `A || B` 表示字节串拼接。
- `U16(x)`、`U32(x)` 分别表示 x 的 2 字节、4 字节网络序编码。
- `HMAC-SHA256(K, M)` 表示以 K 为密钥、M 为消息的 HMAC-SHA256。
- `SHA256(M)` 返回 32 字节摘要。
- 接收端必须先校验帧头 CRC 和负载 CRC，再解释消息内容。

2. UDP 承载

YSP/1 每个 UDP 数据报恰好承载一个 YSP 帧，不允许在一个 UDP 数据报中拼接多个帧。服务端默认监听 UDP/46000；客户端源端口由实现自行选择。

捕获文件可能包含非 YSP 数据报、校验失败的帧、乱序帧及重传帧。接收端不得依赖抓包顺序重组对象。

3. 固定帧头

YSP/1 帧头固定为 28 字节。

偏移	长度	字段	说明
0	2	magic	固定为 ASCII `YS`，即 `0x59 0x53`
2	1	version	固定为 `0x01`
3	1	message_type	消息类型，见第 4 节
4	1	flags	位标志；bit 0 为 RETRANSMIT
5	1	header_length	固定为 28 (`0x1c`)
6	4	session_id	会话标识
10	4	sequence	方向内序号；重传可重复
14	2	stream_id	会话内对象流标识
16	2	fragment_index	分片序号，从 0 开始
18	2	fragment_count	当前逻辑消息的总分片数
20	2	payload_length	帧负载长度，不含 28 字节头
22	2	header_crc16	对字节 0..21 计算 CRC-16/CCITT-FALSE
24	4	payload_crc32	对传输态负载计算 IEEE CRC-32

`CRC-16/CCITT-FALSE` 参数：poly `0x1021`，init `0xffff`，refin false，refout false，xorout `0x0000`。

`CRC-32` 与 IEEE 802.3/zlib 相同。计算结果以无符号 32 位整数处理。

4. 消息类型

值	名称	方向与用途
`0x01`	HELLO	客户端发起会话
`0x02`	WELCOME	服务端接受会话
`0x03`	OPEN	客户端请求对象路径
`0x04`	META	服务端给出对象元数据与处理管线
`0x05`	DATA	服务端发送对象分片
`0x06`	ACK	客户端确认接收进度
`0x07`	FIN	服务端给出传输态对象摘要

4.1 HELLO

负载布局：

偏移	长度	字段
0	16	client_nonce
16	1	client_id_length
17	可变	UTF-8 client_id

HELLO 使用 `stream_id = 0`。

4.2 WELCOME

偏移	长度	字段
0	16	server_nonce
16	4	server_time，Unix 秒

WELCOME 使用 `stream_id = 0`。

4.3 OPEN 与 TLV

OPEN 负载由连续 TLV 组成。每项为 `type:u8 || length:u16 || value:length bytes`。

TLV 类型	名称	值
`0x01`	object_path	UTF-8 对象路径
`0x02`	request_token	8 字节请求标识

未知 TLV 必须按 length 跳过。

4.4 META

偏移	长度	字段	说明
0	4	object_id	对象标识
4	4	encoded_length	三层封装后的完整对象长度
8	4	decoded_length	最终文档长度
12	2	nominal_chunk_size	发送端建议分片大小
14	1	pipeline_count	管线算法 ID 数量
15	1	reserved	必须为 0
16	32	decoded_sha256	最终文档 SHA-256
48	可变	pipeline	`pipeline_count` 个算法 ID

管线按照接收端处理顺序排列。标准对象的管线为：

```
01 02 03 31 30
XTEA-CBC -> RC4-DROP1024 -> ROLX -> BASE85-DECODE -> ZLIB-DECOMPRESS
```

4.5 DATA

DATA 头部中的 `fragment_index` 与 `fragment_count` 可用于缺片检查；实际重组必须以 DATA 负载中的 offset 为准。

偏移	长度	字段	说明
0	4	object_id	必须与 META 一致
4	4	offset	在完整传输态对象中的绝对偏移
8	2	data_length	ciphertext 长度
10	可变	ciphertext	会话级流加密后的分片
10+N	8	data_tag	明文分片的认证标签

重传帧 flags bit 0 为 1。相同 `(session_id, stream_id, object_id, offset)` 的有效分片必须内容一致；接收端只保留一份。

4.6 ACK

负载为 `object_id:u32 || next_offset:u32`。ACK 不参与对象恢复。

4.7 FIN

负载为 `object_id:u32 || encoded_sha256:32 bytes`。摘要针对完成 DATA 解密并按 offset 重组后、尚未剥离内容层的对象。

5. 会话密钥

HELLO、WELCOME 和固定帧头中的字段均为公开输入。定义：

```
master_key = SHA256(
    ASCII("YSP/1 master") ||
    client_nonce ||
    server_nonce ||
    U32(session_id)
)

transport_key = HMAC-SHA256(master_key, ASCII("transport"))
```

同一 session_id 的 HELLO 与 WELCOME 配成一对。不要跨会话混用 nonce。

6. DATA 会话级解密

YSP 采用可随机访问的 HMAC 流。对完整对象绝对位置 p：

```
block_index = floor(p / 32)
in_block    = p mod 32
KS_block    = HMAC-SHA256(
    transport_key,
    ASCII("chunk") || U16(stream_id) || U32(block_index)
)
P[p] = C[p] XOR KS_block[in_block]
```

算法以 DATA 的 offset 为绝对起点，因此分片边界通常不与 32 字节密钥流块对齐。加密与解密操作相同。

解密出分片明文 P 后验证：

```
expected_tag = first_8_bytes(HMAC-SHA256(
    transport_key,
    ASCII("data") || U32(object_id) || U32(offset) || P
))
```

只有标签一致的分片可以进入重组。所有区间 `[0, encoded_length)` 填满后得到传输态对象。

7. YSL1 内容层容器

三种加密算法都使用相同的 32 字节明文层头。层头本身不加密，紧随其后的是 ciphertext。

偏移	长度	字段	说明
0	4	layer_magic	ASCII `YSL1`
4	1	algorithm_id	`0x01`、`0x02` 或 `0x03`
5	1	options	YSP/1 中必须为 0
6	2	layer_header_length	固定为 32
8	4	plaintext_length	本层解密结果长度
12	4	ciphertext_length	紧随层头的密文长度
16	8	nonce	本层 nonce；XTEA 中同时作为 IV
24	4	plaintext_crc32	本层解密结果的 IEEE CRC-32
28	4	layer_header_crc32	对字节 0..27 的 IEEE CRC-32

每剥离一层后都必须检查 `plaintext_length` 与 `plaintext_crc32`。前一层的明文是下一个完整 YSL1 容器，直到 META 管线进入非加密变换。

所有加密层的密钥材料按以下方式导出：

```
layer_key = HMAC-SHA256(
    master_key,
    ASCII("YSP-LAYER") || algorithm_id:u8 || nonce:8bytes
)
```

7.1 算法 0x01：XTEA-CBC

- 使用标准 XTEA，32 轮，delta `0x9e3779b9`。
- 每个 64 位块解释为两个网络序 u32；128 位密钥解释为四个网络序 u32。
- 密钥为 `layer_key` 的前 16 字节。
- CBC 的 8 字节 IV 即层头 nonce。

- 使用 PKCS#7 填充，块大小为 8 字节；即使明文刚好对齐也增加一整块。

单块加密伪代码：

```
sum = 0
repeat 32 times:
    v0 += (((v1<<4 XOR v1>>5) + v1) XOR (sum + key[sum AND 3]))
    sum += 0x9e3779b9
    v1 += (((v0<<4 XOR v0>>5) + v0) XOR (sum + key[(sum>>11) AND 3]))
```

所有算术按 u32 回绕。解密执行逆序减法。

7.2 算法 0x02：RC4-DROP1024

- 使用标准 RC4 KSA 与 PRGA。
- 密钥为完整 32 字节 `layer_key`。
- KSA 后先生成并丢弃 1024 个密钥流字节，再与 ciphertext 异或。
- RC4 加密与解密相同，不使用额外填充。

7.3 算法 0x03：ROLX

令 K 为 32 字节 `layer_key`，N 为 8 字节 nonce。对从 0 开始的每个字节位置 i：

```
r = (K[i mod 32] XOR N[i mod 8]) AND 7
m = K[(i + N[0]) mod 32] XOR ((61*i + N[i mod 8]) mod 256)
C[i] = ROL8(P[i] XOR m, r)
```

解密为：

```
P[i] = ROR8(C[i], r) XOR m
```

当 r 为 0 时，ROL8 与 ROR8 保持字节不变。

8. 非加密内容变换

8.1 算法 0x31：BASE85-DECODE

使用 RFC 1924 字母表的 Base85，无换行、无前缀。Python 参考操作为 `base64.b85decode(data)`。

8.2 算法 0x30：ZLIB-DECOMPRESS

输入是 RFC 1950 zlib 数据流，内部为 DEFLATE。Python 参考操作为 `zlib.decompress(data)`。

9. 推荐接收流程

1. 从 UDP/46000 中寻找 magic 为 YS 的数据报。
2. 校验固定头、长度、header CRC16 与 payload CRC32，丢弃失败帧。
3. 按 session_id 配对 HELLO/WELCOME，计算 master_key 与 transport_key。
4. 解析 OPEN 的 object_path，选择目标 stream_id。
5. 解析 META，记录 object_id、长度、摘要与处理管线。
6. 对每个 DATA 按 offset 生成密钥流，解密并校验 data_tag；去重后按 offset 重组。
7. 可用 FIN 的 encoded_sha256 检查重组结果。
8. 按 META 管线顺序剥离 XTEA-CBC、RC4-DROP1024、ROLX 层。
9. 执行 Base85 解码与 zlib 解压。
10. 用 META 中 decoded_length 与 decoded_sha256 验证最终文档。

10. 实现注意事项

- 抓包顺序、sequence 和 fragment_index 都不能替代 offset。
- payload_crc32 校验的是线上 ciphertext；data_tag 校验的是会话级解密后的分片。
- YSL1 plaintext CRC 校验的是当前层解密结果，包括下一层的完整层头。
- 多个会话可以同时存在；session_id、stream_id 和 object_id 共同确定对象上下文。
- 任何 CRC、长度、标签或摘要失败都应当被视为解析上下文错误或数据损坏，不应静默继续。

Yunsee Stream Protocol 1.0 - End of Specification